1987 PORSCHE 944S · RESTOMOD ELECTRONICS

System Map & Bench Reference

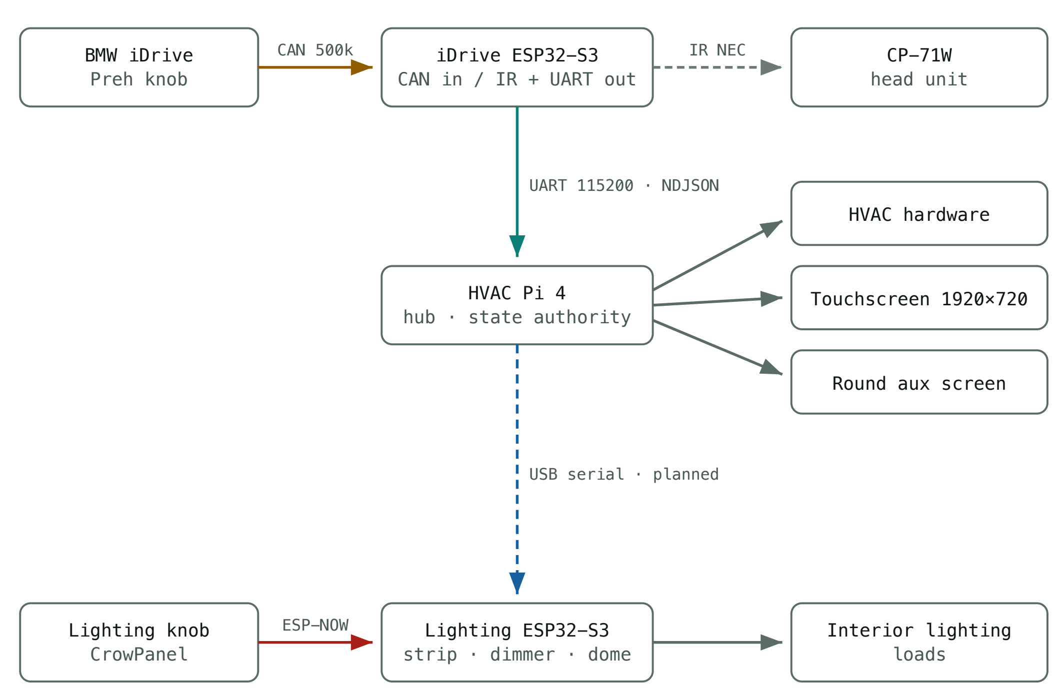
Every module, every interconnect, and the complete I/O map for the HVAC Pi. Pin assignments here are read from source, not from documentation — `hvac_backend.py`, `idrive_controller.ino`, and `pwm_controller.ino`.

`6 modules` `4 ESP32s on the Pi USB hub` `generated 2026-07-30`

01 System map

Link colour is the transport, which is also what you probe it with — a scope on CAN, a terminal on serial, nothing at all on the radio link.

CAN 500 kbps **UART 115200** **USB serial** **ESP-NOW** **IR 38 kHz**



Not in this bench setup. The two T-Display-S3-Long aux gauge panels and the ACC/IGN ignition display are separate projects with no link to this system yet. The iDrive's GAUGE mode is reserved for the gauge panels but currently emits actions nothing consumes.

02 Module inventory

HVAC Pi 4

Hub · climate control · state authority

Host	944HVACPi · user mark
Path	/home/mark/hvac/
Net	eth0 192.168.1.142
Runtime	Python 3.11 · FastAPI · 10 Hz loop
Serves	:8000 dashboard + /ws
Repo	pelosim/944_HVAC

iDrive Controller

Input surface · IR transmitter

Board	Lonely Binary ESP32-S3 N16R8
MAC	D0:CF:13:24:DB:B8
Core	esp32 3.3.10 (~/.arduino-cli-esp32v3)
Flash	tools/flash-via-pi.sh
Version	v1.6.0
Repo	pelosim/idrive-controller

Lighting Output

Strip · dimmer · dome relay

Board	Lonely Binary ESP32-S3 N16R8
MAC	3C:DC:75:40:0B:F0
Core	esp32 2.0.14 (~/.arduino-cli-esp32v2)
Partition	Huge APP (3 MB, no OTA)
Repo	pelosim/Automotive-Lighting-Controller

Lighting Knob

Rotary UI · ESP-NOW only

Board	ELECROW CrowPanel 1.28" round
MCU	ESP32-S3R8 · 240x240 GC9A01
Encoder	CLK 45 · DT 42 · BTN 41
LCD power	GPIO 1 + 2 HIGH before tft.begin()
Stack	LVGL 9.x · TFT_eSPI

03 Interconnects

LINK	TRANSPORT	PHYSICAL	CARRIES	STATUS
iDrive knob → ESP32	CAN 500k	Green=CANH, Green/Org=CANL SN65HVD230 · 120Ω term	ID 0x25B, all 14 inputs	Working
ESP32 → head unit	IR NEC	GPIO17 → 220Ω → LED	5 codes, addr 0x00	Loopback 15/15
ESP32 → Pi	UART	ESP 43→Pi 15 · ESP 44←Pi 14 common GND	NDJSON actions	Working
ESP32 ↔ Pi	USB	/dev/idrive → ttyACM0	console + remote flashing	Working
Pi → lighting board	USB	/dev/lighting	L SET / L ADJ / L GET	Working
Knob ↔ lighting board	ESP-NOW	no wires, shared GND	4-byte packets	Working

Why the iDrive keeps both links. UART carries the data because `/dev/serial0` is stable and survives resets on either end. USB carries power and flashing because `ttyACM` re-enumerates every time the ESP reboots. Moving data onto USB would mean writing reconnect logic for no gain.

04 HVAC Pi — GPIO map

BCM numbering. Relays and solenoids are **active-LOW** — the pin is driven low to energise, so a floating or un-initialised pin leaves them off.

BCM	FUNCTION	DIRECTION	ACTIVE	DRIVES
4	DS18B20 1-Wire bus	bidir	1-Wire	3× temp sensors
5	Blower HI	out	LOW = on	relay

BCM	FUNCTION	DIRECTION	ACTIVE	DRIVES
6	Blower LOW	out	LOW = on	relay
12	Footwell flap IN1	out	HIGH	H-bridge
13	Seat heat — driver	out PWM	2 Hz	MOSFET
14	UART TXD → iDrive RX	out	serial	ESP GPIO44
15	UART RXD ← iDrive TX	in	serial	ESP GPIO43
16	Defrost flap IN1	out	HIGH	H-bridge
18	A/C clutch → MS3	out	LOW = on	relay RLY1
19	Heat valve solenoid	out	LOW = on	relay RLY2
20	Defrost flap IN2	out	HIGH	H-bridge
21	Footwell flap IN2	out	HIGH	H-bridge
23	Blend flap → COLD	out	HIGH	H-bridge
24	Blend flap → HOT	out	HIGH	H-bridge
25	Seat heat — passenger	out PWM	2 Hz	MOSFET
26	Fresh-air solenoid	out	LOW = on	relay RLY3
2 / 3	I²C SDA / SCL	bidir	I²C	ADS1115 @ 0x48

ANALOG FEEDBACK — ADS1115 @ 0X48

CH	MEASURES	0%	100%	WATCHDOG
A0	Blend flap position	225 mV	4090 mV	cut after 8.0 s <1.5% travel
A1	Defrost flap position	225 mV	4090 mV	as above
A2	Footwell flap position	225 mV	4090 mV	as above
A3	spare	—	—	—

TEMPERATURE — DS18B20 ON GPIO 4

SENSOR	ROM ID	RANGE SHOWN
Mixing chamber (duct)	`000000bd3d51`	`32–180 °F`
Exterior	`000000be5d11`	`-20–120 °F`
Interior / cabin	`000000bbdd26`	`20–140 °F`

> **IDs carry no 28– prefix.** The w1thermsensor library strips it. Adding it back gives three sensors that all read zero and a dashboard that looks alive but is lying.

SERIAL PORTS

DEVICE	RESOLVES TO	PEER	BAUD	NOTES
`/dev/serial0`	`ttyAMA0`	iDrive UART	`115200`	needs `dtoverlay=disable-bt`
`/dev/idrive`	`ttyACM1`	iDrive USB	–	udev by MAC D0:CF:13:24:DB:B8
`/dev/lighting`	`ttyACM0`	Lighting board	`115200`	udev by MAC 3C:DC:75:40:0B:F0
`/dev/gauges1`	`ttyACM2`	Gauge panel A (master)	–	console + flashing; no Pi protocol yet
`/dev/gauges2`	`ttyACM3`	Gauge panel B (slave)	–	named only — no firmware assembled

> **Two boot-config settings are load-bearing.** `dtoverlay=disable-bt` moves `/dev/serial0` onto the real PL011. Without it you get the mini-UART, whose baud tracks the VPU core clock and corrupts under load — which the 10 Hz control loop guarantees.
>
> And `console=serial0,115200` must stay *out* of `cmdline.txt`, or the kernel console fights the backend for the same port. Backups sit beside both files as `.bak`.

05 ESP32 I/O

BOARD	GPIO	FUNCTION	NOTES
iDrive Controller	**4**	CAN TX (CTX)	`SN65HVD230 — never MCP2551`
	8	CAN RX (CRX)	`120Ω across CANH/CANL`
	17	IR LED	`220Ω series`
	18	IR receiver	`bench loopback only`
	43 / 44	UART to Pi	`TX / RX`
	48	NeoPixel	`mode colour feedback`
Lighting Output	**4**	Illumination sense	`INPUT_PULLUP, LOW = night`
	5	NeoPixel data (CH1)	`330Ω · 4× Flora V2`
	6	MOSFET PWM (CH2)	`IRLZ44N · 5 kHz · 8-bit`
	7	Dome relay	`active HIGH`

> **GPIO 5, 6, 7, 15, 16 on the iDrive board are free.** They hold the unused SWC resistor ladder (`OUT_SWC0`), kept as a fallback in case the radio's fixed ladder is ever worth chasing.

06 Wire protocols

IDRIVE → PI · NEWLINE-DELIMITED JSON

```
{"mode":"HVAC","action":"TEMP_UP","count":2}
mode    RADIO | HVAC | ILLUM | GAUGE
action  TEMP_UP TEMP_DOWN FAN_UP FAN_DOWN HVAC_TOGGLE
        HVAC_MODE_NEXT HVAC_MODE_PREV AUX_SWAP
        LIGHT_BRIGHTER LIGHT_DIMMER LIGHT_TOGGLE
        LIGHT_SCENE_NEXT LIGHT_SCENE_PREV  MODE_ENTER
        VOL_UP VOL_DOWN MUTE NEXT PREV   (mirror only — IR does the work)
count   detents in this frame, clamped 1..12
```

PI ↔ LIGHTING BOARD · TEXT OUT, JSON BACK

```
L SET <ch> <val>     absolute
L ADJ <ch> <delta>   relative — what the Pi actually sends
L GET                 request a state report

ch  1 = strip brightness   2 = MOSFET dimmer
    3 = dome relay         5 = palette index 0–9

{"src":"illum","ch1":200,"ch2":128,"color":3,"relay":1,"night":0}
```

LIGHTING KNOB ↔ OUTPUT BOARD · ESP-NOW, 4 BYTES

```
[ header ][ channel ][ value ][ channel XOR value ]

0xAA  knob → output   0x01 ch1 · 0x02 ch2 · 0x03 relay · 0x05 colour
0xBB  output → knob   0x04 day/night
```

Lighting commands are relative, never absolute. The Pi cannot know current brightness — the round knob may change it at any moment — so it sends `L ADJ 1 +24` and the output board decides what that means, then reports the truth back. The board was already the owner of those values; this keeps it that way, and is the only thing stopping the two controllers from drifting apart.

07 Bench bring-up

Ordered so each step depends only on ones already passed. If a step fails, stop — later results will be misleading.

1 **Power the Pi alone**

No ESP boards, no loads. Confirm the backend comes up and the dashboard serves.

`systemctl is-active hvac-backend → active · http://944hvacpi.local:8000 renders`

2 **Verify sensors before any actuator**

All three DS18B20s should read plausible room temperature, and the ADS1115 should answer.

`/api/state → onewire_ok true, ads_ok true, three distinct temps`

3 **Flap travel, one at a time**

Drive each flap end to end and watch its feedback percentage track. The 8-second no-progress watchdog should cut the motor if a flap is disconnected.

`position sweeps 0→100 · disconnected flap sets *_fault true and stops`

4 **Relays and solenoids**

Blower HI/LOW, A/C clutch, heat valve, fresh-air. All active-LOW — listen for the click.

`each energises on command, none energised at rest`

5 **Seat heaters**

2 Hz PWM to the MOSFETs. Check duty at LOW/MED/HIGH (33/66/100%).

`visible slow pulsing, no buzz from the MOSFET board`

6 **iDrive on CAN, bench-powered**

120Ω termination fitted, grounds common. It needs one physical button press to wake on the bench.

`serial shows PRESS lines for all 14 inputs`

7 **iDrive UART into the Pi**

Turn the knob in HVAC mode and watch the setpoint follow on screen.

`idrive_last_s non-zero · clockwise raises setpoint`

8 **IR at the head unit**

Aim the LED at the CP-71W's IR window from close range first.

`volume moves · if it works close but not from the dash, that is drive current — put the LED behind a 2N3904, do not touch the firmware`

9 **Lighting board on USB**

Flash it with the core 2.0.14 toolchain, plug into the Pi, capture its USB serial, then enable the udev line.

`/dev/lighting appears · backend logs "lighting link up"`

08 Known traps

Each of these cost real debugging time. They share a shape: the symptom points somewhere other than the cause.

SYMPTOM	ACTUAL CAUSE
LED lags, drops encoder events, looks like flaky hardware — then works perfectly the moment a terminal is opened	USB CDC blocks on write with no host draining it, up to ~2 s per printf. Fix is `Serial.setTxTimeoutMs(0)`. Present on both ESP32 boards.
Encoder dies after ~20 s of turning, buttons keep working	Byte 2 of `0x25B` is not a two-state flag — it latches at 0x81 during sustained rotation. Never gate rotation on it.
Setpoint numerals shove the layout as the value changes	Orbitron has no tabular figures — "1" is 0.391 em against 0.834 em for "0". `tabular-nums` is silently ignored; fixed cells are the fix.
Serial link works, then reports "multiple access on port"	`console=serial0` left in cmdline.txt — the kernel console holds the same port.
Flashing an ESP over the Pi works sometimes, fails others	ModemManager probes new ttyACM devices with AT commands. udev rule ignores VID 303a.

SYMPTOM	ACTUAL CAUSE
Flashing a board over the Pi dies partway with "No more data to read from the serial port"	The backend holds that port. It grabs `/dev/lighting` the instant udev creates the symlink, then fights esptool for the device and leaves a half-written app partition. `flash-via-pi.sh` now stops and restarts the backend around the write.
Dashboard "still looks old" after a rebuild	Browser cache. Hard-refresh, or restart the kiosk.

09 Harness mapping — factory sheet 5

Read from *Wiring Diagram Type 944 / 944 turbo / 944 S, Model 87, Sheet 5*. The Pi substitutes for the **Heating and A/C control switch**, so almost everything below lands on that unit's A and B connectors.

GERMAN COLOUR ABBREVIATIONS

ABBR	GERMAN	COLOUR	ABBR	GERMAN	COLOUR
SW	`schwarz`	black	**BR**	`braun`	brown (ground)
WS	`weiß`	white	**GR**	`grau`	grey
RT	`rot`	red	**BL**	`blau`	blue
GN	`grün`	green	**GE**	`gelb`	yellow
LI	`lila`	lilac / violet	**RS**	`rosa`	pink

`1,0 SW/RT` reads as 1.0 mm² black with a red tracer — base colour first, stripe after the slash, cross-section in front.

FLAP ACTUATORS — THE BULK OF THE WORK

Each flap is one DC motor plus one feedback potentiometer in a single 5-pin connector. **Pins 4 and 5 are the motor; 1, 2 and 3 are the pot, with pin 3 as the wiper.** Handily, the wiper is the only un-striped wire in each family.

FLAP / PI PINS	PIN	ROLE	WIRE	TERMINAL
Defrost GPIO 16 / 20 ADS A1	**4**	motor	1,0 GN/SW	B11
	5	motor	1,0 WS/SW	B12
	2	pot end	0,5 SW/BL	B5
	3	**wiper**	0,5 SW	B7
	1	pot end	0,5 SW/RT	A6
Footwell GPIO 12 / 21 ADS A2	**4**	motor	1,0 GN/LI	B9
	5	motor	1,0 GN/WS	B10
	2	pot end	0,5 GN/BL	B3
	3	**wiper**	0,5 GN	B8
	1	pot end	0,5 GN/RT	A5
Blend / temp mix GPIO 23 / 24 ADS A0	**4**	motor	1,0 GR/SW	A10
	5	motor	1,0 GR/GN	A11
	2	pot end	0,5 GR/RT	A7
	3	**wiper**	0,5 GR	A12
	1	pot end	0,5 GR/BL	B6

Pot excitation is now your job. The factory control head fed these dividers; the Pi must. Your scaling constants — 225 mV at 0%, 4090 mV at 100% — imply roughly 5 V across the element, so the ADS1115 needs to be on 5 V too. On 3.3 V the top of travel clips and every reading above ~80% is wrong.

On the blend flap the /RT and /BL ends read swapped relative to the other two. That may be deliberate (flap sense) or my reading of the scan — ohm it out before trusting the direction.

SOLENOIDS AND A/C

FUNCTION	PI	WIRE	TERMINAL
Heat valve solenoid	`GPIO 19`	`1,0 SW`	`A4`
Recirc / fresh-air solenoid	`GPIO 26`	`1,0 SW/GN`	`A3`
Solenoid common ground	–	`0,5 BR`	`chassis`
A/C clutch request → MS3	`GPIO 18`	`via A/C relay G17`	`sheet col. A–B`

BLOWER — READ THIS BEFORE WIRING RELAYS

The factory blower is **four speeds**, not two. A resistor pack (0.25 / 0.4 / 1.0 / 2.5 Ω in series) is tapped by the fresh-air blower switch:

SWITCH POS.	TAP	WIRE	TERMINAL	RESULT
1	4	`1,5 BR/WS`	`C1`	slowest — full resistance
2	3	`1,5 WS/GE`	`C6`	—
3	2	`2,5 WS`	`C3`	—
4	1	`2,5 BR/SW`	`C5`	fastest — least resistance
–	motor	`2,5 BR`	`ground`	blower return

Your software models two speeds, the car has four. `fan_speed` is OFF / LOW / HI driving `GPIO 5` and `GPIO 6`, but the harness offers four taps. Pick two and accept the coarser control, or add two more relays and widen `fan_speed` — either is fine, but decide before wiring rather than discovering it at the bench.

These are the fat wires on the sheet — 2.5 mm² carrying blower current. Size the relay contacts for the motor's stall draw, not its running draw.

WHAT YOU DO *NOT* NEED TO CONNECT

FACTORY ITEM	WHY IT IS REDUNDANT
Inside / outside / mixed-chamber temp sensors	Replaced by your three DS18B20s on GPIO 4

FACTORY ITEM	WHY IT IS REDUNDANT
Heating and A/C control switch	The Pi *is* this unit now — you are wiring into its connectors
Fresh air blower switch	Replaced by relays under Pi control
Seat heater time relay and 4-way switches	Bypassed — the Pi drives the elements through MOSFETs on GPIO 13 / 25
Evaporator icing protection	Series safety in the A/C circuit — leave it in place, do not bypass

Before applying power on the bench. Ohm every pot end-to-end (expect a few kΩ) and wiper-to-end (should sweep smoothly as the flap moves by hand). A pot wired where a motor belongs will be destroyed the instant an H-bridge drives it.

Colours here are transcribed from a scan. Buzz each one out at the connector before you commit — the sheet is the map, the meter is the territory.

10 Address registry

Every identity in the system, in one place. The ESP32-S3 reports its MAC as its USB serial, so the same string identifies a board on the bench, on the hub, and over ESP-NOW — which is what makes a single registry possible at all.

ESP32 BOARDS

BOARD	MAC / USB SERIAL	PORT	TOOLCHAIN	FIRMWARE
iDrive controller Lonely Binary N16R8	D0:CF:13:24:DB:B8	/dev/idrive	core 3.3.10	idrive_controller v1.6.0
Lighting output Lonely Binary N16R8	3C:DC:75:40:0B:F0	/dev/lighting	core 2.0.14	pwm_controller
Gauge panel A — master T-Display-S3-Long #1	68:EE:8F:48:19:9C	/dev/gauges1 /dev/gauges	core 2.0.14	gauge_bench

BOARD	MAC / USB SERIAL	PORT	TOOLCHAIN	FIRMWARE
Gauge panel B — slave `T-Display-S3-Long #2`	`68:EE:8F:48:18:D4`	`/dev/gauges2`	`core 2.0.14`	`none yet`
Lighting knob `CrowPanel 1.28"`	`not on the hub`	`—`	`core 2.0.14`	`display_controller`

> **The two gauge panels are near-sequential MACs from one batch** — …`19:9C` and …`18:D4`. Only the full serial tells them apart, which is exactly why every board is pinned by serial rather than by vendor. Adding the hub already reshuffled `ttyACM` numbering once.

HOSTS AND SERVICES

NAME	ADDRESS	NOTES
HVAC Pi — hostname	`944HVACPi`	ssh alias `pi944`, user `mark`
HVAC Pi — eth0	`192.168.1.142`	DHCP; has moved before — prefer the hostname
HVAC Pi — wlan0	`192.168.1.240`	DHCP
mDNS	`944HVACPi.local`	unreliable — has cached stale leases and resolves to a dead IPv6 record; use `ssh -4`
Dashboard	`http://944HVACPi.local:8000`	WebSocket at `/ws`, REST at `/api/state`

PROTOCOL IDENTIFIERS

BUS	IDENTIFIER	MEANING
CAN	`0x25B`	iDrive — all 14 inputs, 500 kbps
CAN	`0x202 / 0x563 / 0x0AA / 0x130 / 0x440 / 0x560`	keep-alives the iDrive needs to stay awake
CAN	`0x5E7 → 0x273`	init handshake, reply format unconfirmed

BUS	IDENTIFIER	MEANING
IR	`NEC addr 0x00`	cmds 0x02 vol+ · 0x98 vol– · 0xE2 mute · 0x90 next · 0xE0 prev
ESP-NOW	`0xAA / 0xBB`	knob→output / output→knob, 4-byte packets
I²C	`0x48`	ADS1115 — on the Pi *and* on gauge panel A
1-Wire	`bd3d51 / be5d11 / bbdd26`	DS18B20 duct / exterior / cabin — no 28– prefix

Panel B has no firmware and must not be flashed with panel A's. `slave_app.h` is an app layer waiting to be dropped into a copy of LilyGo's `lvgl_demo`; that sketch has not been assembled. Panel A's `gauge_bench` drives an ADS1115 that panel B does not have — `flash-via-pi.sh gauges2` refuses for exactly this reason.